

name: yadah_music_ai

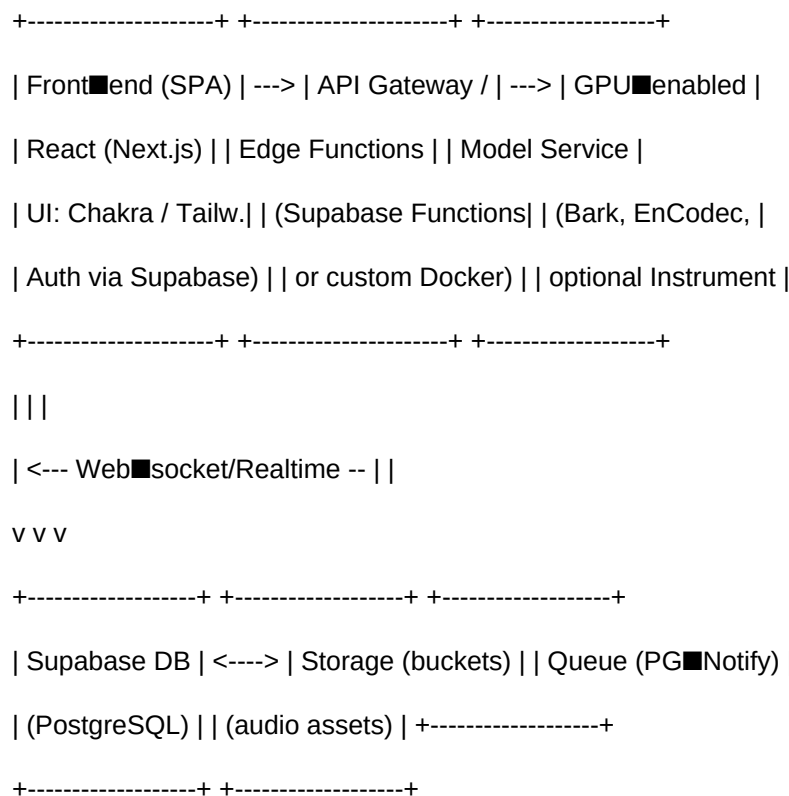
description: "High-level tech brainstorm for the Yadah Music AI project, architecture, DB schema, API contracts, Supabase integration, and comparison to Suno.ai."

■ Quick Overview

Goal – Build a web-first AI music-generation SaaS that mirrors Suno AI's capabilities (text-to-song, optional lyric generation, vocal + instrument stems) while using **Supabase** for auth, storage, and real-time data.

Core insight – Suno's stack is a **hierarchical token-based transformer pipeline** (Bark → Chirp → EnCodec). Replicating that pipeline (or reusing the open-source **Bark** model) gives you a solid technical foundation without building a new generative model from scratch.

1 ■ System Architecture (high-level)



- **Frontend** – Next.js (React) with TypeScript, UI library (Chakra UI + Tailwind), Supabase JS client for auth & realtime.

- **Backend** – Supabase Edge Functions (Node.js) or a lightweight Express/Koa service hosted on a GPU-enabled provider (Render, Fly.io, Railway). Functions receive a generation request, call the model service, store resulting WAV/MP3 in Supabase Storage, and return a signed URL.

- **Model Service** – Deploy the open-source **Bark** (semantic → coarse → fine) + **EnCodec** decoder. Either host on a dedicated GPU VM (Dockerised) or use a 3rd party inference platform (Replicate, HuggingFace Inference). For instrument stems you can add a second model (e.g., **MusicGen** or a custom **Chirp**-like transformer).

- **Data layer** – Supabase PostgreSQL with Row Level Security (RLS). Stores users, song projects, generation jobs, metadata about audio assets, and payment info if you later monetize.

- **Realtime** – Supabase realtime channel pushes generation status (queued, processing, completed) to the UI, enabling a progress bar without polling.

2 Database Schema (core tables)

| Table | Fields | Purpose |

|-----|-----|-----|

| **users** | id (uuid PK), email, created_at, metadata | Supabase Auth users (RLS ties to auth.users). |

| **projects** | id (uuid PK), owner_id (FK → users.id), title, description, created_at, updated_at | Logical grouping of a song (can have multiple tracks). |

| **tracks** | id (uuid PK), project_id (FK), title, duration_ms, audio_url (text), status (enum), created_at | Stores each generated piece (vocals, instrument, remix). |

| **generation_requests** | id (uuid PK), project_id (FK), prompt (text), lyrics (text, nullable), model_params (jsonb), status (queued|running|failed|done), log (text), started_at, finished_at | Queue for background processing; used by workers to update status. |

| **payments** (optional) | id, user_id, amount_cents, currency, status, created_at | For SaaS billing if you charge per generation. |

Indexes – Primary key on id, plus BTree indexes on owner_id (users), project_id (projects/tracks/requests). Use a GIN index on model_params if you need to filter by GPU settings.

3 API Routing Contracts (REST – JWT via Supabase)

| Method | Endpoint | Auth | Payload | Response |

|-----|-----|-----|-----|-----|

| POST | /api/v1/projects | ■ | {title, description} | {project} |

| GET | /api/v1/projects/:id | ■ | – | {project, tracks[]} |

| POST | /api/v1/generate | ■ | {project_id, prompt, lyrics?, style?, length_sec?} | {request_id, status_url} |

| GET | /api/v1/generation/:id/status | ■ | – | {status, progress, audio_url?} |

| GET | /api/v1/tracks/:id | ■ | – | {track, audio_url} |

| POST | /api/v1/auth/login | – (Supabase) | – | {access_token, refresh_token} |

| POST | /api/v1/auth/logout | – | – | {ok} |

Authentication – All protected routes require the Supabase JWT (Authorization: Bearer <jwt>). Use Supabase RLS policies to enforce that a user can only access their own projects, tracks, and generation_requests.

4 ■ Similarities & Differences vs. Suno AI

| Area | Suno AI | Yadah Music AI (proposed) |

|-----|-----|-----|

| **Model pipeline** | Multi-stage transformer (Bark → EnCodec → Chirp) | Same pipeline – reuse **Bark** (open-source) + **EnCodec**; optionally add an instrument model (MusicGen or a fine-tuned Chirp). |

| **Open-source vs. proprietary** | Bark is open; rest of stack closed. | End-to-end stack can be **fully open-source** (frontend, backend, DB, model) – gives you control and lower licensing risk. |

| **Frontend stack** | React (Chakra UI, Tailwind) + Cloudflare CDN. | Same – React/Next.js + Chakra UI + Tailwind, served via Vercel/Cloudflare. |

| **Auth & storage** | Internal auth, custom DB (likely Postgres). | **Supabase** provides auth, Postgres, storage, and realtime out-of-the-box – faster MVP. |

| **Scalability** | Runs on dedicated GPU clusters; uses Cloudflare for edge caching. | Use Supabase Edge Functions (CPU) for orchestration and a separate GPU service (Docker VM/Replicate) for heavy inference. Horizontal scaling via queue (PGNotify) + Supabase’s autoscaling. |

| **Data flywheel** | User-generated songs feed continuous fine-tuning. | We can build the same loop: store generated songs (with user consent) in an S3 bucket, periodically fine-tune the Bark model on this curated dataset (requires GPU training pipeline). |

| **Pricing model** | Free tier + paid credits per song. | Start with **free tier** on Supabase + pay-as-you-go GPU inference (Replicate pricing) – easy to expose a credit system later. |

Takeaway

Core similarity: both rely on a hierarchical token pipeline (semantic → coarse → fine) decoded by EnCodec. Re-using **Bark** gives you the same “semantic-first” advantage.

Key differentiator: you have **full control** over the entire stack via Supabase, open-source model code, and a custom UI – you can iterate faster, add features (e.g., collaborative editing, stem export), and avoid vendor lock-in.

5 ■ Immediate Next Steps (MVP roadmap)

Scaffold the front-end – npx create-next-app@latest yadah-music-ui --ts. Add Chakra UI & Tailwind, configure Supabase client.

Create Supabase project – enable Auth, Database, and Storage bucket (audio). Write the DB schema from section 2 (run via Supabase SQL editor).

Deploy a small GPU inference service – simplest: a **Replicate** endpoint for *Bark* (public). Write a tiny wrapper in a Supabase Edge Function that forwards the request, polls for completion, and stores the WAV in Supabase Storage.

Implement API routes – `/api/v1/generate` that writes a row in `generation_requests`, triggers the Edge Function, and returns a polling URL.

Realtime status – use Supabase's realtime channel (`pg_notify`) to push status updates to the UI.

Add optional lyrics generation – call a small LLM (e.g., OpenAI's `gpt-4o-mini` or Claude) to generate verses when a user supplies only a theme.

Testing & CI – write unit tests for API handlers, integration tests for the queue, and a GitHub Actions workflow that spins up a cheap GPU runner for end-to-end tests.

Future enhancements – add stem export, allow users to upload custom instrument samples, implement a paid-credits system, and eventually **fine-tune Bark** on the curated dataset you collect.

6 ■ Risks & Mitigations

| Risk | Impact | Mitigation |

|-----|-----|-----|

| **GPU cost** – inference at scale can get pricey. | High OPEX if you go viral. | Use a **pay-per-use** model (Replicate credits), and add a server-side **rate limit** per user. Cache identical prompts (hash → stored audio). |

| **Latency** – multi-stage pipeline adds seconds per song. | Poor UX if generation > 30 s. | Pre-warm the model service, run the **semantic** stage on CPU (fast), then offload coarse/fine to GPU only when needed. Show a progress UI with estimated time. |

| **Audio quality artifacts** (metallic, clipping). | Users may abandon the product. | Fine-tune on a clean dataset, add a post-process DSP step (normalisation, low-pass filter). |

| **Intellectual property / content policy** – generating copyrighted styles could violate policies. | Legal risk. | Enforce a **content policy filter** on prompts (e.g., block brand names, explicit copyrighted lyrics). |

| **Supabase limits** – storage quota, database row limits. | Service may hit limits as usage grows. | Enable **bucket versioning** and **row-level archiving** (move old tracks to cold storage after 30 days). |

7 ■ TL;DR Summary

Adopt **Bark + EnCodec** (open-source) for the generation core – mirrors Suno's hierarchical token pipeline.

Build the web app with **Next.js + Chakra UI**, using **Supabase** for auth, PostgreSQL, storage, and realtime.

Deploy a **GPU-enabled inference service** (self-hosted Docker VM or Replicate) that the Supabase Edge Function calls.

Define a simple DB schema (users, projects, tracks, generation_requests) and a REST API with JWT auth.

Leverage Suno's strengths (data flywheel, token-based generation) while keeping the whole stack under your control.

Start with the MVP steps above, then iterate on instrument models, fine-tuning, and monetisation.

Sources

- Suno Tech Stack – Technologies Used by Suno | Web Radar
(<https://the-web-radar.com/companies/suno.com>)

- How Suno AI Creates Music: Technical Deep Dive into AI Music Generation 2025
(<https://musicgeneratorai.io/posts/how-does-suno-ai-create-music>)

- Suno: The AI studio turning prompts into platinum-level tracks (<https://www.todayin-ai.com/p/suno>)

- Comparative Analysis of Suno and Udio AI Music Generation Systems
(<https://comparative-analysis-of--7h7m9pw.gamma.site/>)

- Technological Moats - Suno AI Business and Product Deep Dive (<https://oboe.com/learn/suno-ai-business-and-product-deep-dive-y9k2c3/technological-moats-suno-ai-business-and-product-deep-dive-0>)